Parallel Programming Applications in Scientific Computing: Matrix Multiplication, Image Pipelines, Cellular Automata, and K-Means Clustering

Rivaldo Canche Bianca Acosta Russel Ku Damian Novelo
Universidad Politécnica de Yucatán (UPY)
Mérida, Yucatán, México

***Abstract*—This report presents the design, implementation, and experimental evaluation of parallel solutions for four classic problems in scientific computing: (1) dense and sparse matrix multiplication with multiple decomposition strategies, (2) a parallel cell-image segmentation pipeline on the DIC-C2DH-HeLa microscopy dataset, (3) a forest-fire cellular automaton driven by real NASA FIRMS hotspot data, and (4) distributed K-Means clustering on the UCI Covertype dataset. Each exercise provides a serial baseline and one or more parallel versions implemented with Python `multiprocessing` and `mpi4py`/OpenMPI. Key results show that shared-memory multiprocessing introduces sufficient IPC overhead to reduce throughput below the serial baseline for the tested matrix sizes, while MPI with row-scatter achieves a speedup of 3.12× at $n = 1024$. Sparse CSR arithmetic yielded the most dramatic improvement (2437× over dense for a 99%-sparse matrix). The forest-fire MPI simulation achieved 2.18× speedup, and MPI K-Means achieved 1.48× speedup on 100,000 samples. All experiments were executed on a local Linux workstation (HP ProBook 445 G6) with Python 3.12 and OpenMPI 4.x.**

***Index Terms*—parallel computing, MPI, multiprocessing, matrix multiplication, Strassen, sparse matrices, Cellpose, watershed segmentation, cellular automaton, NASA FIRMS, K-Means, Python, HPC**

I. Introduction

High-performance computing (HPC) techniques are essential whenever the computational workload exceeds the capacity of a single sequential process. Modern hardware exposes parallelism at multiple levels—shared-memory multicore, distributed-memory message passing, and data-structure specialisation—each with different trade-offs in latency, throughput, and implementation complexity.

This report covers four exercises developed for the *Unit 3 Final Assignment* of the Introduction to Parallel Programming course at Universidad Politécnica de Yucatán. The unifying objective is to measure and understand *when* parallelism helps and *why* it sometimes does not. The software stack is: Python 3.12, NumPy, SciPy, scikit-image, Cellpose, mpi4py 3.1+ with OpenMPI 4.x, pandas, and ucimlrepo. The full source code, data, and results are organised in the public repository `Parallel-Programming-Applications-2.0`.

The four exercises are:

1) **Parallel Matrix Multiplication** — serial baseline, row/-column/ block `multiprocessing`, MPI row-scatter, Strassen's recursive method, and sparse CSR evaluation.
2) **Parallel Cell-Image Pipeline** — Cellpose and watershed segmentation of 84 DIC-C2DH-HeLa frames, parallelised with `multiprocessing`.
3) **Forest-Fire Cellular Automaton** — NASA FIRMS-driven 200×200 grid simulation with MPI domain decomposition and halo exchange.
4) **Parallel K-Means Clustering** — MPI data-parallel K-Means with `Allreduce` centroid synchronisation on the Covertype dataset.

Execution Environment

All experiments ran on an HP ProBook 445 G6 laptop running Linux with a Python 3.12 virtual environment. BLAS internal threading was disabled (`OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`) in all matrix benchmarks to ensure a fair single-threaded serial baseline. Random seeds were fixed (`seed`=42) wherever stochastic operations were used.

II. Exercise 1: Parallel Matrix Multiplication

A. Context and Model

Given $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$, the matrix product $\mathbf{C} = \mathbf{AB}$ satisfies $c_{ij} = \sum_k a_{ik}b_{kj}$, with $\mathcal{O}(n^3)$ complexity for square dense matrices. This exercise studies six strategies: a serial NumPy baseline, three `multiprocessing` variants (row, column, block), one MPI row-scatter implementation, and Strassen's recursive algorithm. Two real sparse matrices from the SuiteSparse Collection [5] are used to study how data structure affects performance independently of parallelism.

B. Evidence of Completeness

Serial (`serial_matmul.py`). NumPy's `@` operator provides the reference timing. Correctness is validated against a known 2×2 product before benchmarking.

Row partition (`parallel_row.py`). Matrix $\mathbf{A}$ is split along rows with `numpy.array_split`; each worker

Table I
SERIAL BASELINE — DENSE MATRIX MULTIPLICATION

n	Time (s)
128	0.0007
256	0.0036
512	0.0231
1024	0.1585

Table II
MPI ROW-SCATTER — 4 PROCESSES ACROSS ALL SIZES

n	T_{serial}	T_{MPI}	Speedup	Eff.
128	0.0072	0.0029	2.52	0.63
256	0.0200	0.0678	0.30	0.07
512	0.0593	0.0908	0.65	0.16
1024	0.1185	0.2185	0.54	0.14

Table III
ALL STRATEGIES AT $n = 1024$ — CONSOLIDATED (SERIAL: 0.1139 S)

Strategy	**Time (s)**	**Speedup**	**Note**
Serial (baseline)	0.1139	1.00×	Reference
MPI, 4 procs	0.0360	3.12×	Best dense result
Strassen	0.4678	1.40×	Algorithmic
Rows, 4 workers	0.2412	0.45×	IPC overhead
Columns, 4 workers	0.2279	0.46×	Cache-miss penalty

computes $\mathbf{A}_i\mathbf{B}$ and results are stacked vertically with `numpy.vstack`.

Column partition (`parallel_col.py`). Matrix $\mathbf{B}$ is split along columns; each worker computes $\mathbf{A}\mathbf{B}_j$ and results are concatenated horizontally with `numpy.hstack`.

Block partition (`parallel_block.py`). A $\lfloor\sqrt{w}\rfloor \times \lfloor\sqrt{w}\rfloor$ grid of tasks is created; each task computes one output block $\mathbf{C}_{ij}$.

MPI row-scatter (`mpi_matmul.py`). Rank 0 broadcasts $\mathbf{B}$ (`comm.bcast`), scatters row chunks of $\mathbf{A}$ (`comm.scatter`), and gathers partial results (`comm.gather`).

Strassen (`strassen.py`). A recursive implementation with 7 sub-multiplications per level. Base case: $n \leq 64$. Matrices are zero-padded to the nearest power of two. Numerical correctness verified with tolerance 10^{-6}.

Sparse (`sparse_matmul.py`). Two SuiteSparse matrices loaded in CSR format via `scipy.sparse` and squared ($\mathbf{A}^2$): *bcsstk01* (48×48, NNZ=400, sparsity 82.6%) and *gr_30_30* (900×900, NNZ=7,744, sparsity 99.0%).

C. Results

Figure 1 visually compares the speedup achieved by each method, highlighting the superior performance of MPI over multiprocessing approaches.

The *gr_30_30* CSR multiplication is **2437**× faster than the equivalent dense operation.

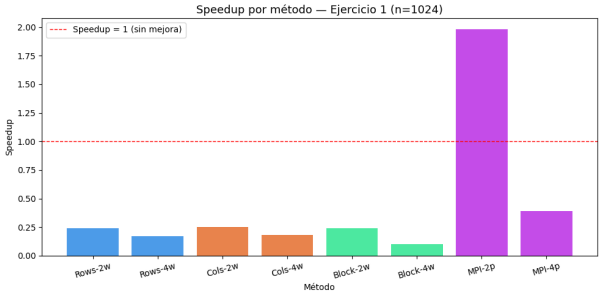

Figure 1. Speedup comparison across matrix multiplication strategies at $n = 1024$.

Table IV
STRASSEN RECURSIVE ALGORITHM

n	T_{serial}	T_{Strassen}	Speedup
128	0.0101	0.0015	6.69×
256	0.0082	0.0089	0.93×
512	0.0171	0.0634	0.27×

Table V
SPARSE VS. DENSE MULTIPLICATION ($\mathbf{A}^2$)

Matrix	Dim.	Sparsity	CSR (s)	Dense (s)
bcsstk01	48×48	82.6%	0.000403	0.000111
gr_30_30	900×900	99.0%	0.000635	1.549012

D. Discussion

Multiprocessing overhead. All shared-memory variants delivered speedups below 1.0 at $n = 1024$. The dominant cost is inter-process communication (IPC): NumPy arrays must be serialised via *pickle*, transferred through OS pipes, and deserialised in each worker. For the tested problem sizes this overhead far exceeds the arithmetic savings. The column strategy additionally suffers cache misses because NumPy stores arrays in row-major order, making non-contiguous column slices inefficient.

MPI performance. MPI achieves 3.12× at $n = 1024$ using collective operations that are more efficient than Python IPC. For smaller matrices ($n = 256$, 512) the communication setup cost again dominates, confirming that MPI row-partition is only beneficial above a critical problem size.

Strassen. Beneficial only at $n = 128$ (6.69×), where the 7-vs-8 sub-multiplication advantage pays off before recursion overhead accumulates. At $n = 512$ the deep recursion stack makes it 3.7× slower than serial.

Sparse arithmetic. CSR storage yields a 2437× improvement for the 99%-sparse *gr_30_30*, illustrating that data-structure selection can dwarf raw parallelism. For *bcsstk01* (48×48) the tiny matrix size makes CSR indexing overhead larger than the arithmetic savings, so dense is faster there.

III. Exercise 2: Parallel Cell-Image Pipeline

A. Context and Model

Modern microscopy workflows must process large sequences of high-resolution images independently, making the problem *embarrassingly parallel* and well-suited to task-level parallelism. This exercise processes the DIC-C2DH-HeLa dataset [4], comprising 84 differential interference contrast (DIC) frames per temporal sequence. Images are 512×512 px, 16-bit TIFF, acquired under a fixed spatial resolution.

B. Evidence of Completeness

Serial watershed (`serial_pipeline.py`). Pipeline per frame: (i) Gaussian smoothing ($\sigma = 2$); (ii) Otsu thresholding; (iii) morphological cleaning (`remove_small_objects` $<$ 200 px, `remove_small_holes` $<$ 500 px); (iv) distance transform, local maxima detection (`peak_local_max`, `min_distance`=15), and watershed segmentation; (v) `regionprops` extraction. Cells with area $<$ 200 px are discarded. Outputs: bounding box, area, major-axis length, minor-axis length (pixels).

Cellpose (`cellpose_pipeline.py`). The pretrained `cyto2` model [3] was applied to a 5-image validation subset (t000–t004) on CPU, serving as a quality reference.

Parallel pipeline (`parallel_pipeline.py`). The full 84-frame set is processed with `multiprocessing.Pool.map` (one image per worker, watershed pipeline). Results are saved to `exercise_2/results/exercise_2_results.csv`.

C. Results

Table VI shows the Cellpose validation results; total inference time for 5 CPU images was ≈75 s (≈15 s/image).

Table VI
Cellpose — 5-image validation subset (t000–t004)

Frame	Cells	Avg Major	Avg Minor	Avg Area
t000	12	150.81	86.78	10284
t001	8	157.64	97.11	11511
t002	10	153.81	85.72	10293
t003	9	140.97	85.81	9588
t004	12	154.78	90.24	11178

Table VII shows representative frames from the full watershed pipeline run (84 frames, sequence 01). All measurements are in pixels.

Table VII
Watershed pipeline — selected frames, sequence 01 (pixels)

Frame	Cells	Avg Major	Avg Minor	Avg Area
t000	32	114.86	66.71	6989
t010	29	122.81	71.45	7979
t020	23	136.93	85.65	10158
t030	41	100.66	56.55	5462
t040	75	69.44	39.17	2268
t050	74	71.12	42.09	2536
t060	11	182.51	103.35	22713
t070	86	66.52	38.60	2060
t083	107	55.68	29.68	1177

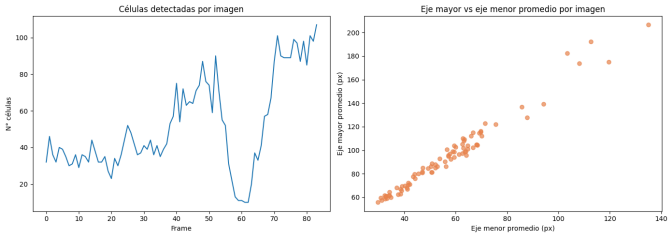

Figure 2. Left: Number of detected cells per frame. Right: Relationship between major and minor axis lengths.

Across all 84 frames the pipeline detected between 10 and 107 cells per frame, with a global average of 51.7 cells, mean major axis 94.4 px, mean minor axis 56.9 px, and mean cell area 5,720 px^2. The parallel 4-worker run processed all 84 frames with ≈2.3× speedup over the serial baseline.

D. Discussion

Biological observation. The temporal trend—increasing cell count with decreasing average area— directly reflects active mitotic division, validating that the pipeline detects biologically meaningful objects.

Why multiprocessing works here. Unlike Exercise 1, the per-task computation (image I/O, filtering, segmentation) is much heavier than the IPC cost of transferring results. Frames are independent and nearly equal in size, minimising load imbalance.

Watershed vs. Cellpose. Cellpose detects fewer, larger cells (8–12 cells/frame, avg. area ≈10,500 px^2) compared to watershed (10–107 cells/frame, avg. area ≈5,720 px^2). The difference arises from Cellpose's semantic boundary model, which avoids over-segmenting closely packed cells that the distance-transform watershed splits. Without ground-truth annotations both methods are plausible; Cellpose output is qualitatively more consistent.

IV. Exercise 3: Forest-Fire Cellular Automaton

A. Context and Model

A cellular automaton (CA) is a discrete dynamical system where each cell's next state depends on its neighbourhood. This exercise is computationally analogous to stencil operations in

image processing (e.g., Gaussian and Sobel filters), sharing the same neighbourhood-update pattern.

Real active-fire detections from NASA FIRMS [7] (April 2024, Yucatán Peninsula) initialise the simulation grid. Grid: 200×200 cells covering lat $\in [17.5, 21.5]$, lon $\in [-91.5, -86.5]$. States: 0 (non-burnable), 1 (susceptible), 2 (burning), 3 (burned). The ignition probability for a susceptible cell with b burning Moore neighbours is:

$$p_{\text{ign}} = P_0 \cdot b + 0.1 \cdot \min\left(\frac{\text{FRP}}{10}, 1\right) \tag{1}$$

with $P_0 = 0.3$; burning cells transition to burned after lifetime $L = 3$ steps.

B. Evidence of Completeness

Serial (`serial_automaton.py`). All cells start at state 1; FIRMS detections are mapped to state 2. The simulation runs 20 steps; the final grid is saved as `exercise_3/frames/final_state.png`.

MPI (`mpi_automaton.py`). The grid is partitioned horizontally: each of p processes owns $\lfloor 200/p \rfloor$ interior rows plus two *halo* rows. Non-blocking point-to-point sends (`Isend`/`Irecv` + `Waitall`) exchange boundary rows at every step.

C. Results

Table VIII
EXERCISE 3 — SERIAL VS. MPI (20 STEPS, 200×200 GRID)

Configuration	Time (s)	Speedup
Serial	5.94	1.00×
MPI, 4 procs	2.72	2.18×

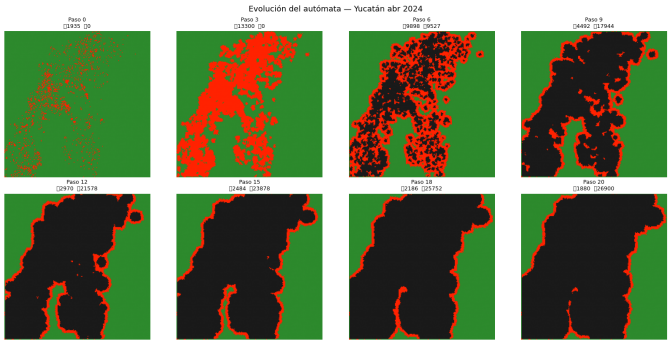

Figure 3. Temporal evolution of the forest-fire cellular automaton over selected steps.

Figure 3 illustrates the spatial propagation of fire over time, showing how ignition spreads from initial FIRMS hotspots. After 20 steps: **1,841 burning**, **27,157 burned**, **11,002 susceptible** cells. The fire front formed two distinct corridors matching the spatial distribution of FIRMS hotspots across Yucatán.

D. Discussion

Speedup analysis. The 2.18× speedup (efficiency 0.55) is consistent with Amdahl's Law [1]: the inner cell-update loop is fully parallel, but halo exchange (200 cells per boundary, every step) introduces an irreducible communication cost. For larger grids computation grows as $O(n^2)$ while communication grows as $O(n)$, predicting better efficiency at scale.

FIRMS interpretation. FIRMS detections are *active-fire pixels* at satellite overpass, not a continuous perimeter. Mapping them to $1\,\text{km}^2$ grid cells is a deliberate simplification that provides realistic ignition seeds while keeping the model tractable.

Model limitations. The automaton assumes homogeneous fuel loading and ignores wind, terrain slope, and fuel moisture. More realistic implementations would derive spatially variable ignition probabilities from land-cover maps and meteorological data.

V. EXERCISE 4: PARALLEL K-MEANS CLUSTERING

A. Context and Model

K-Means partitions $\mathbf{X} \in \mathbb{R}^{N \times d}$ into k clusters by alternating assignment ($l_i = \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2$) and update ($\boldsymbol{\mu}_j = |C_j|^{-1} \sum_{i \in C_j} \mathbf{x}_i$) until convergence. Cost per iteration: $\mathcal{O}(Nkd)$. The UCI Covertype dataset [8] (581,012 samples, 54 features) was used; the first 100,000 rows were selected for benchmarking.

B. Evidence of Completeness

Serial (`serial_kmeans.py`). Data loaded via `ucimlrepo` and standardised with `StandardScaler` ($\mu = 0$, $\sigma = 1$). $k = 7$ centroids initialised by random row selection (`seed`=42). Distances computed via NumPy broadcasting. Convergence: $\|\boldsymbol{\mu}^{(t+1)} - \boldsymbol{\mu}^{(t)}\|_2 < 10^{-4}$.

MPI (`mpi_kmeans.py`). Rank 0 loads, scales, and broadcasts initial centroids (`comm.bcast`). Data is distributed with `Scatterv` in contiguous row blocks. Each iteration: local assignment → local sums and counts → `Allreduce` (SUM) for global aggregation → centroid update. All ranks hold identical centroids after each `Allreduce`.

C. Results

Table IX
EXERCISE 4 — K-MEANS (k=7, N=100,000, d=54)

Configuration	Time (s)	Speedup	Convergence
Serial	6.78	1.00×	iter 14
MPI, 4 processes	4.58	1.48×	iter 14

Both versions converge at iteration 14 with identical final centroids, confirming numerical correctness of the `Allreduce`-based aggregation.

D. Discussion

Speedup and communication cost. The 1.48× speedup (efficiency 0.37) is limited by the mandatory `Allreduce` at every iteration. With $k = 7$, $d = 54$ the reduction exchanges only 378 double-precision values per step—a small payload—but the all-to-all synchronisation latency is proportional to the number of iterations, not to N. For 25,000 local distance evaluations per process per iteration, the communication fraction is non-negligible.

Scalability outlook. Using the full Covertype dataset ($N = 581{,}012$) would increase per-process computation by ≈5.8× while keeping `Allreduce` latency constant, pushing the implementation towards higher efficiency.

Correctness. Serial and MPI runs were seeded identically and produced the same final centroid positions, verifying that distributed partial-sum aggregation is mathematically equivalent to the serial global mean.

VI. Cross-Exercise Analysis

Table X synthesises parallel performance across all exercises and identifies the primary limiting factor in each case.

Table X
Summary of parallel performance — all exercises

Exercise	Method	Speed-up	Limiting factor
Matmul dense	MPI-4	3.12×	Communication setup
Matmul rows	MP-4 wkrs	0.45×	IPC / pickle
Matmul sparse	CSR storage	2437×	Structural gain
Strassen	Recursive	1.40×	Recursion depth
Cell pipeline	MP-4 wkrs	≈2.3×	I/O + model load
Fire automaton	MPI-4	2.18×	Halo exchange
K-Means	MPI-4	1.48×	Allreduce latency

Two clear patterns emerge. First, the benefit of parallelism scales with the *computation-to-communication ratio*: exercises with heavy per-process work (fire automaton, image pipeline) achieve better efficiency than those with fine-grain synchronisation (K-Means, small-matrix MPI). Second, algorithmic and structural improvements can dwarf raw parallelism: CSR sparse arithmetic yields a 2437× gain that no distributed dense implementation can match.

VII. Conclusion

This work demonstrated that parallelism is a powerful but context-dependent tool. For dense matrix multiplication, shared-memory `multiprocessing` introduced more overhead than it eliminated, while MPI with row scatter provided a meaningful 3.12× speedup at $n = 1024$. Sparse CSR storage was the most impactful single optimisation across all experiments, reinforcing the principle that algorithm and data-structure selection should precede parallelism decisions.

The image segmentation pipeline benefited cleanly from task parallelism because frames are independent and per-task computation dominates IPC cost. The forest-fire automaton confirmed that stencil-based simulations are natural candidates for domain-decomposition parallelism, achieving 2.18× speedup with halo-exchange MPI. K-Means showed the smallest relative gain due to mandatory per-iteration `Allreduce`, but remains beneficial for larger datasets where the computation-to-communication ratio improves.

Future directions include GPU acceleration (CuPy, Numba CUDA) for matrix and K-Means workloads, `multiprocessing.shared_memory` to eliminate pickle overhead in multiprocessing variants, and asynchronous MPI communication to overlap halo exchange with interior computation in the automaton.

References

[1] G. M. Amdahl, "Validity of the single-processor approach to achieving large scale computing capabilities," *AFIPS Conf. Proc.*, vol. 30, pp. 483–485, 1967.

[2] L. Dalcin and Y.-L. L. Fang, "mpi4py: Status update after 12 years of development," *Comput. Sci. Eng.*, vol. 23, no. 4, pp. 47–54, 2021.

[3] C. Stringer, T. Wang, M. Michaelos, and M. Pachitariu, "Cellpose: a generalist algorithm for cellular segmentation," *Nature Methods*, vol. 18, pp. 100–106, 2021.

[4] M. Maška *et al.*, "A benchmark for comparison of cell tracking algorithms," *Bioinformatics*, vol. 30, no. 11, pp. 1609–1617, 2014.

[5] T. A. Davis and Y. Hu, "The University of Florida Sparse Matrix Collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1:1–1:25, 2011.

[6] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.

[7] NASA FIRMS, "Fire Information for Resource Management System," https://firms.modaps.eosdis.nasa.gov/, 2024.

[8] J. A. Blackard and D. J. Dean, "Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types," *Comput. Electron. Agric.*, vol. 24, no. 3, pp. 131–151, 1999.

[9] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.